

Qianyue Wang - 400385783
Bowen Wang - 400325152
Mian Chen - 400385299
Tianyao Zhang - 400375124
Ethan Suitor - 400411786
Owen Marshall - 400470427
Yicheng Liu - 400359801

Group 5
GhostDrive (Twin)

System Design

Project Title: *Human-in-the-Loop Cognitive Digital Twin of an Autonomous Scale Vehicle*

Team: Group 5

Supervisor: Dr. Istvan David

Table of Contents

Table of Contents	2
Revision History	3
1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions and Acronyms	5
1.4 Operating Domain Assumptions	6
1.5 Technology Selection Rationale	6
2. System Variables	7
2.1 Monitored Variables	7
2.2 Estimated Variables	8
3. System Architecture & Overview	11
3.1 Context Diagram	11
3.2 System Breakdown	12
3.3 Function Decomposition	13
3.4 FSM	14
4. Module Hierarchy	15
5. Detailed Sub-System Design (Software-Digital Twin)	15
5.1 M1: Hardware Abstraction Layer (HAL)	15
5.2 M2: Perception	18
5.3 M3: Visualizer & User Control	21
5.4 M4: Navigation	24
5.5 M5: Lower-Level Controller	26
6. Module Internal Design	27
6.1 Electrical Schematics	28
6.2 Bill of Materials (BoM)	28
6.3 Hardware Abstraction Layer	29

7. Traceability Matrix.....	30
8. Undesired Event Handling	31
8.1 Normal Behaviour.....	31
8.2 Wi-Fi Disconnects.....	32
8.3 Camera Occluded.....	32
8.4 Simulink Model Crashes	32
9. Appendices.....	33

Revision History

Table 1: Revision History

Date	Version	Changes	Reason of Change
Dec 21, 2025	1.0		
Mar 20, 2026	2.0	Added 1.4. operating domain assumptions Updated 7. Traceability Matrix - Updated new requirements - Updated decision mapping(deliverable)	
Mar 19, 2026		Added 1.5 rational	
Mar 20, 2026	2.0	Reorganized the placement and layout of the variable tables, and refined the table contents for clarity.	

1. Introduction

1.1 Purpose

Our project is developing a human follower robot. Our robot is a scaled down car like mobile robot equipped with a camera and a lidar.



Figure 1: The follower robot

We are developing a digital twin system to run on human follower companion. Its primary purpose is to detect, track and follow a designated human by means of sensing and interpreting the surroundings. The physical twin collects and communicates information about its surroundings to the digital twin, which in turn performs computationally expensive tasks to direct the physical twin in its behavior.

The purpose of this document is to define the hardware and software design for the system. It will outline the hardware and software components; then break them down into their respective subsystems to describe the expected behavior.

1.2 Scope

This document defines the hardware and software design for the Human-in-the-Loop Cognitive Digital Twin system. Its scope includes:

- The system architecture and boundaries for the Physical Twin, Digital Twin, and User Cockpit.
- The specification of all monitored inputs and controlled outputs for the system.
- The decomposition of the system into core software and hardware subsystems.
- The detailed design of each subsystem.
- The handling of undesired events, such as communication loss or sensor failure.

- A traceability matrix linking this design back to the system requirements.

1.3 Definitions and Acronyms

Table 2: Definition and Acronyms

Term	Definition
DT	Digital Twin, combination of hardware and software physically separate from the physical twin. Provides a service of performing computations which may be too expensive for the physical twin to perform in a real time setting.
PT	Physical Twin, physical agent which interacts with the environment through various sensors and motors. Behaviour can be manually controlled or autonomous.
Simulink	A block diagram modeling environment within MATLAB. It is used to visually design and test the system's control logic.
Target person	Designated human the system is required to follow.
Mode	Behaviour mode the physical twin operates in.
Following distance	Desired distance to maintain from the target person.
Gesture	Hand gesture which the target person shows to the physical twin to indicate desired behaviour mode.
Obstacle	Any object or person within the physical twin's close proximity which may prevent safe movement.
ROS	Robot Operating System, key part of the software environment, the main communication backbone between the physical and the digital twin
YOLO	You Only Look Once, Object detection algorithm used to identify person and gesture perception
LiDAR	Light Detection and Ranging, sensor that output point cloud that is used for obstacle detection and mapping
FSM	Finite State Machine, mainly used to identify the logic between changing gesture states

Dashboard	The Graphical User Interface of the system(GUI) that is used to monitor the digital twin, displaying the video, kinematics(velocity, acceleration and steer angle) and as well as the simulated surroundings
-----------	--

1. 4 Operating Domain Assumptions

A1. Indoor Low-Speed Operation

The vehicle is assumed to operate on indoor, flat, normal temperature environment environments such as hallways, labs, or open rooms. Outdoor terrain, stairs, steep slopes, rain, and high-speed operation are outside the intended operating domain.

A2. Sensor and Communication Availability

The design assumes the camera, LiDAR, and ROS communication links are available during nominal operation. Failures of these channels are handled as abnormal conditions and cause STOP behaviour rather than continued autonomous following.

A3. Single Authorized Gesture Source

Gesture commands are assumed to be issued by the intended target user rather than arbitrary bystanders in the environment.

A4. Target Visibility During Follow

The follow behaviour assumes that the designated target person is within the camera field of view often enough for perception to refresh target identity and target position.

Note: These assumptions informed later design choices related to speed limiting, target-loss handling, sensor timeout detection, and safe-stop behaviour.

A5. Target Visibility During Normal Follow

Safe battery operation is assumed during normal system operation. Battery shutdown/fault handling is outside the current core control-software scope.

1. 5 Technology Selection Rationale

The digital twin system is being developed to serve as a human follower companion. Its primary purpose is to detect, track and follow a designated human by means of sensing and interpreting the surroundings.

A digital twin architecture is selected in this project to decouple sensing, computation, and control between the physical and virtual systems. This design choice significantly reduces the

computational burden on the embedded platform, which typically has limited processing power and energy constraints. By offloading intensive tasks such as object detection and trajectory planning to an external computing unit, the system achieves improved real-time performance, flexibility, and scalability. Additionally, this architecture enables easier debugging, monitoring, and future upgrades without modifying the physical hardware.

Alternative approaches, such as fully onboard processing, were considered; however, these were not selected due to limitations in processing capability, increased power consumption, and reduced ability to iterate rapidly during development. The digital twin approach therefore provides a more efficient and modular solution for prototyping and system validation.

For perception, a YOLO-based object detection model is adopted due to its balance between detection accuracy and real-time inference speed. In comparison to traditional feature-based methods or heavier deep learning models, YOLO offers faster processing with acceptable accuracy, making it suitable for dynamic environments where low latency is critical. This ensures that the system can reliably detect and track a human target while maintaining stable control performance.

Furthermore, the use of a camera-based perception system is chosen over alternative sensing methods such as LiDAR due to its lower cost, lighter weight, and ease of integration with existing vision-based algorithms. While LiDAR provides higher depth accuracy, it is not necessary for this application where relative positioning and visual tracking are sufficient.

The digital twin is a system that runs outside of the vehicle (e.g. on a laptop or in the cloud). The physical twin collects and communicates information about its surroundings to the digital twin, which in turn performs computationally expensive tasks to direct the physical twin in its behavior. The physical twin is the physical human follower vehicle.

2. System Variables

2.1 Monitored Variables

Variable Name	Description	Units	Usage
M_Scan	2D Lidar Scan	(rad, m)	Obstacle detection and safety

M_RGB	Raw Image from camera feed	Pixels	People / gesture perception
M_Point	Point cloud returned from depthcam/3d lidar – might be implemented for future	m	Environment mapping / obstacle distance estimation
M_Depth	Grey scale depth image returned	Pixels, m	Person distance and obstacle detection
M_Velocity	Monitored speed from the sensor	M/s	Vehicle control and safety monitoring

Table 1: Variable Model - Monitored Variables

2.2 Estimated Variables

Variable Name	Description	Units	Usage
M_DistanceApart	Distance between vehicle and target person	m	Follow mode distance control
M_PersonAngle	Relative angle to the target person	rad	Heading control to keep vehicle facing the target person
M_TargetPosition	Target person position estimate	m, m	Path planning / digital twin update
M_YOLOPerson	Bounding box / detection result for people from the perception module.	[bbox, string, number]	Person detection bounding box from the vision perception module
M_YOLOGesture	Detected gesture label from the perception module (call, like, ok, fist, palm).	[bbox, string, number]	Detected gesture label from the vision perception module

2.3 Controlled Variables

Variable Name	Description	Range	Units	Usage
C_Velocity	Ros command of velocity output	[-1, 1]	m/s	Main speed command used in Follow / Stop behaviour
C_Steering	ROS Commanded steering angle	$[-\frac{\pi}{4}, \frac{\pi}{4}]$	Radian	Heading control to keep the vehicle facing the target person or along the desired path
C_Stop	Emergency stop command if obstacle within the minimum safety distance	[0, 1]	Boolean	Forces the vehicle into a safe stop state, overrides other commands
C_Mode	High-level operating mode command	[0, 5] (discrete)	N.A	Selects overall system behaviour
C_Target	Identifier of the target person	[0, 1]	Boolean	Chooses which detected person is used for follow control and display
C_Distance	The set following distance behind the user	[0.5, 2]	Meters	Setpoint for Follow-mode distance control

Table 2: 4 Variable Models - Controlled Variables

2.4 System Constants

Variable Name	Value	Units	Justification
CONST_MaxV	1.0	m/s	Limit indoor speed for safety.
CONST_MaxW	1.0	rad/s	Avoid unstable, overly sharp turns.

CONST_SafeDist	0.5	m	Minimum clearance to reduce collisions.
CONST_TargetTimeout	1.0	s	Treat missing target as lost and stop.
CONST_LidarTimeout	0.5	s	Detect LiDAR loss and fall back safely.
CONST_YOLOMin	0.8	N.A	Ignore low-confidence detections.
CONST_Sample	0.5	s	Simulink sample time

Table 3: 4 Variable Models - Constants

3. System Architecture & Overview

3.1 Context Diagram

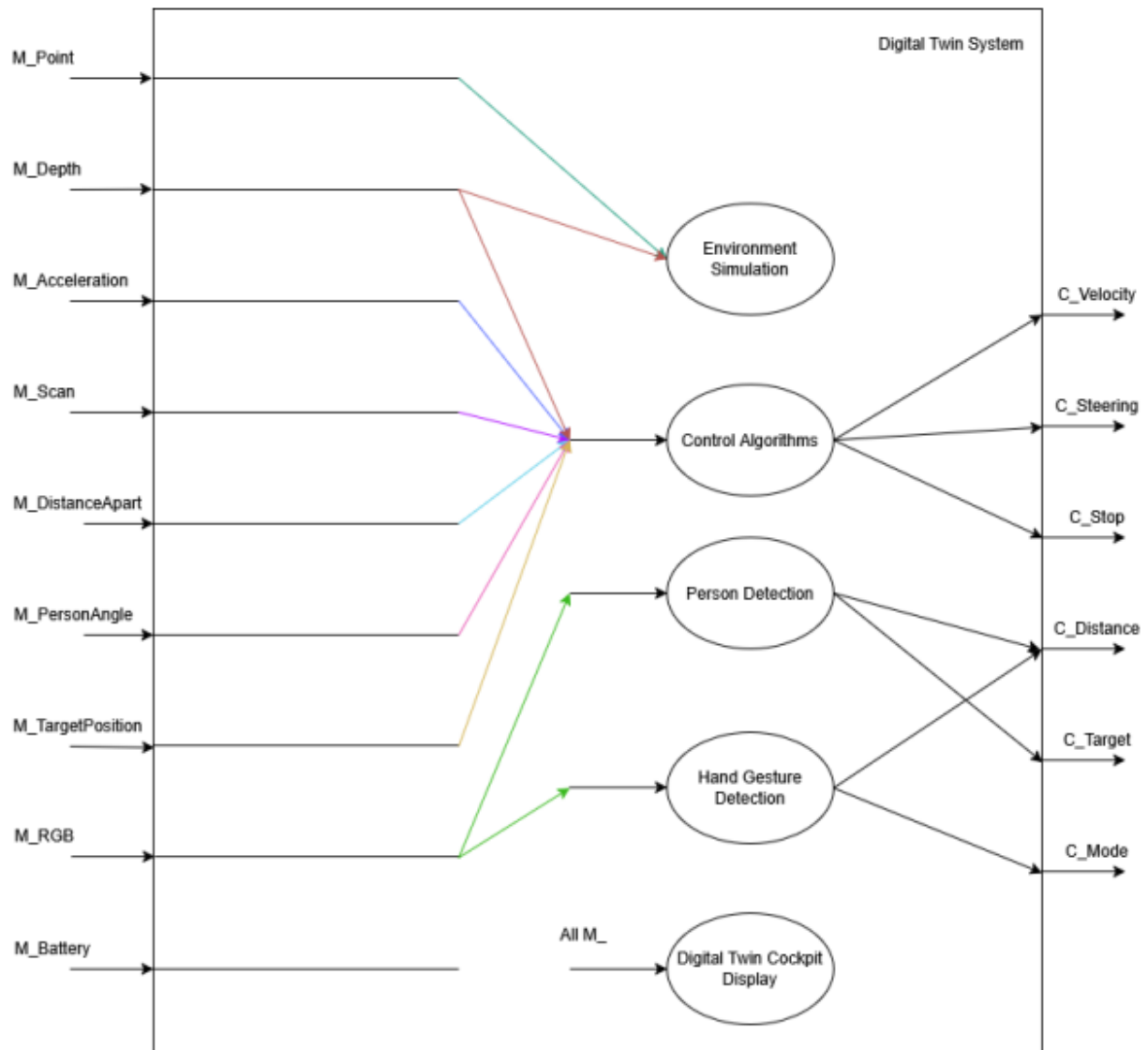


Figure 2: Context Diagram

3.2 System Breakdown

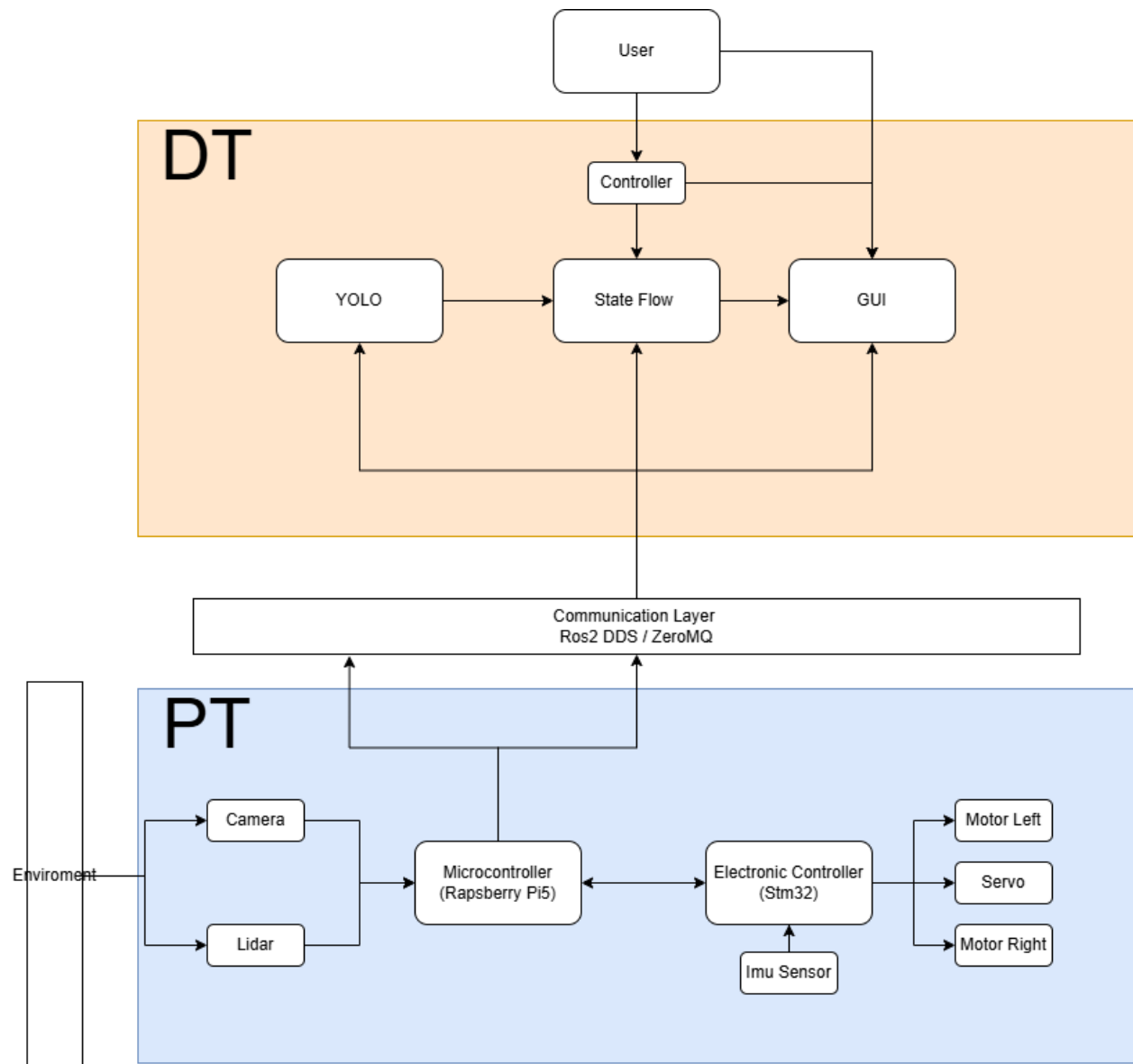


Figure 3: High-level System Breakdown

3.3 Function Decomposition

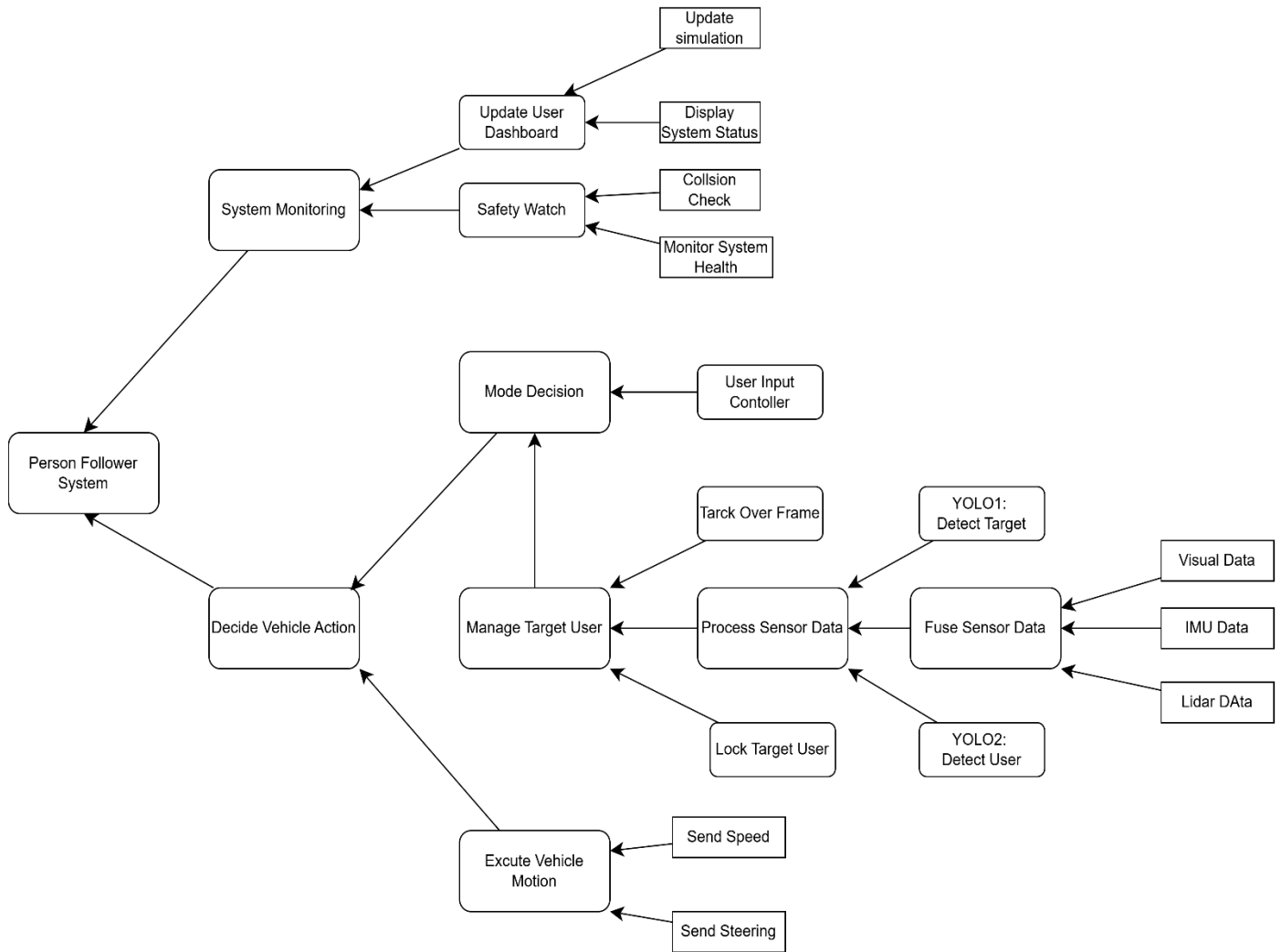


Figure 4: Function Decomposition Diagram

3.4 FSM

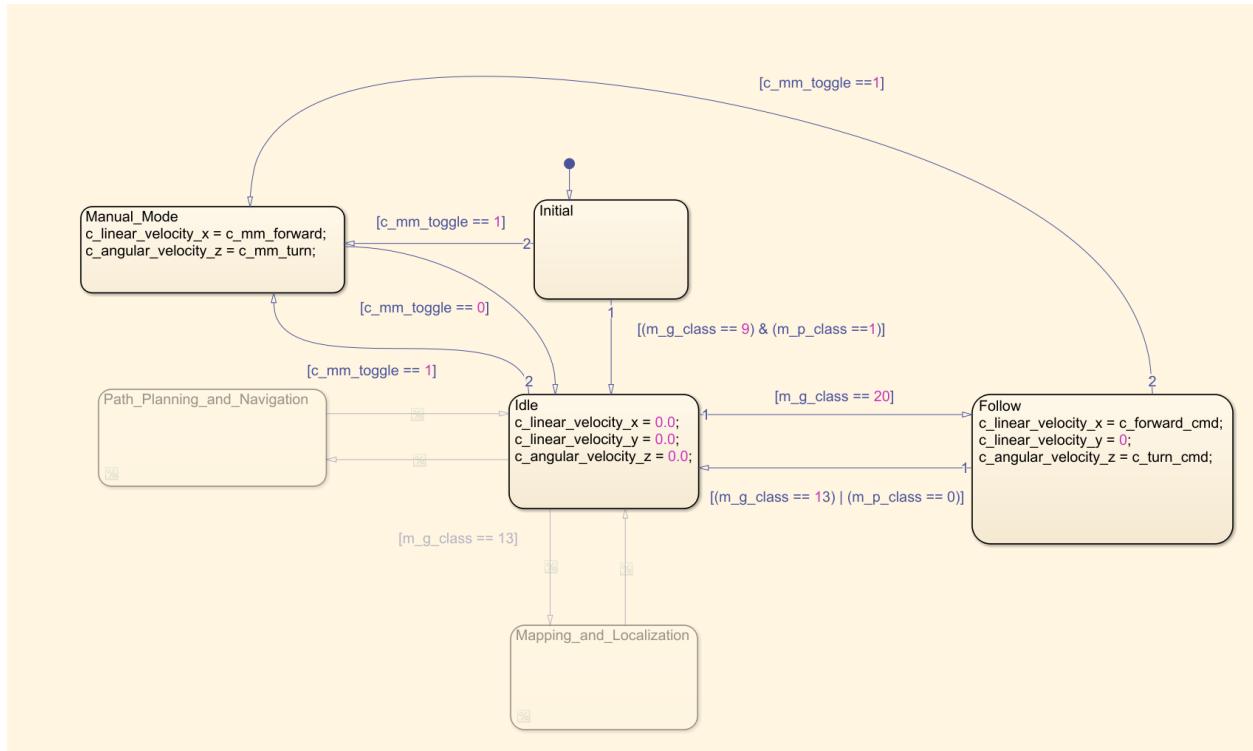


Figure 5: FSM This is the FSM that is within the “State Flow” Module in figure 3

Table 3: FSM variable & description

Variable Name	Description
C_linear_velocity_x	The linear output velocity of the car
C_angular_velocity_z	The steering speed and direction of the car
C_linear_velocity_y	Always 0, there is no y movement is just a place holder to keep the twist matrix format of Ros syntax
C_mm_forward	User control speed
C_mm_turn	User control steer
C_mm_toggle	Boolean indictor of manual mood on/off
M_g_class	Detected gesture

M_p_class	Boolean indicator of locked target
-----------	------------------------------------

4. Module Hierarchy

Figure 3: The Module Structure of the DT

5. Detailed Sub-System Design (Software-Digital Twin)

Decomposition Strategy

The software components are divided based on their responsibilities and using a need-to-know flow of information. For example, the hardware abstraction layer serves as a black box with raw robot sensor data as the input and converted calculation ready data as the output.

The perception module for instance, which takes as input several formatted data inputs from the hardware abstraction layer, need not know how the raw sensor data was converted. Additionally, the granularity of the module's responsibilities was determined such that the modules have a clear purpose, and would make isolating a problem relatively easy, i.e. not overloading a module with several distinct tasks.

5.1 M1: Hardware Abstraction Layer (HAL)

1. Functionality

The HAL system serves as the bridge for communication between the physical hardware sensors and the ROS software. It initialize the sensor drivers, reads the raw data streams(I2C, SPI, UART)(most of it is done internally through the ROS ToolBox) from the Lidar, Camera, and encoders/IMU, and convert them to standard SI units, afterwards publish through ROS node under relevant topics

2. Internal Design

- Hides: Low level communication information such as the specific register addresses of the Lidar, the baud rates of the serial connections, the raw integer values from the ADCs, and the specific hardware API calls.
- Implementation: C++ ROS 2 Lifecycle Nodes.

3. Input / Output Variables

I/O	Variable Name	Data Type	Description	Sources / Destinations
In	hw_Lidar_Signal	Serial Stream	Raw byte stream from Lidar hardware.	Hardware
In	hw_Camera_Signal	CSI/USB Stream	Raw pixel data buffer.	Hardware
In	hw_IMU_Signal	I2C Bus	Raw register values (Accel/Gyro)	Hardware
In	hw_MCU_Signal	UART stream	Battery level in volts	Hardware
Out	M_Scan	Float Array [Distance, Angle]	Normalized distances in meters, angle in degrees	To M2, M3, M4
Out	M_RGB	Image vector	RGB Image Stream(640 x 480)	To M2, M3
Out	M_Depth	Image Vector	Grey Scale Depth Image(640x480)	To M2
Out	M_Velocity	Float	Processed velocity data stream into form of float	To M3
Out	M_Acceleration	Float	Processed acceleration data stream into form of float	To M3
Out	M_Battery	Integer	The Current Battery Status	To M3

Behavior and Range Logics

Variable Name	Input Condition	Output Behavior
hw_Lidar_Signal	distance < 0.1m	0.0m (Too close/Blind spot).
hw_Lidar_Signal	distance > 12.0m	12.0 (Clamped to Max Range).

hw_Camera_Signal	fails/timeout	Output: Trigger System_Error flag; Publish black image frame.

4. Module Interface specification

Method Name	Inputs	Outputs	Method Description
readLidar	hw_Lidar_Signal	M_scan	Convert raw lidar byte stream into distance and angle measurements
readRGB	hw_Camera_Signal	M_RGB	Convert raw compressed rgb data stream into formatted RGB frame
readDepth	hw_Camera_Signal	M_Depth	Converts raw compressed depth camera data into formatted grayscale frame
readIMU	hw_IMU_Signal	M_velocity, M_Acceleration	Converts raw velocity and acceleration data from IMU data stream into float format for further processing
readBattery	hw_MCU_Signal	M_Battery	Convert ADC signal to battery percentage

5. Module Internal Design

Since we are utilizing simulink with several toolboxes, these will mostly be presented in terms of corresponding simulink blocks which achieve the desired functionalities.

Block level design

ROS 2 Subscriber Blocks

- Raw compressed RGB camera topic
- Raw compressed depth camera topic
- Raw compressed LIDAR topic
- Battery status topic (possibly replaced with serial communication block)

Data Conversion Blocks

- Byte formatting for LIDAR data
- Image formatting block for condensed RGB image in form of matrix
- Image formatting block for grayscale depth image in form of matrix
- Scaling and conversion factor blocks

Data Flow

ROS2 Camera Topic -> Image formatting block -> M_RGB

ROS2 Depth Topic -> Image formatting block -> M_Depth

ROS2 Lidar Topic -> Byte formatting -> M_Scan

ROS2 Battery Topic -> Scaling or conversion factor block -> M_Battery

5.2 M2: Perception

1. Functionality

This module processes visual data to identify the navigation target (e.g., the target user or specific hand gesture) and interprets human gestures for control commands. It runs the neural network inference model - YOLO.

Gesture processing in M2 is performed in two stages. First, gesture detections are filtered so that only gestures associated with the currently tracked user are considered. Second, if multiple valid gesture candidates remain, only the highest-confidence gesture is forwarded to downstream decision logic. The confidence will be evaluated by pretrained YOLO models.

2. Internal Design

- Hides: The YOLOv8 Neural Network architecture, the specific weights file, and the non-maximum suppression (NMS) algorithms used to filter bounding boxes, as well as the sensor fusion between the lidar and image.

3. Input / Output Variables

I/O	Variable Name	Data Type	Description	Sources / Destinations
In	M_RGB	Image Vector	Input RGB image	← M1
In	M_Scan	Float vector [distance, angle]	Lidar Scanned Data	← M1
In	M_Depth	Image Vector	Grey scale depth image	← M1
Out	M_YOLOPerson	[Bounding box, confidence]	YOLO output of person detected	→ M3
Out	M_YOLOGesture	Integer	The current command ID	→ M4
Out	M_Objects	[Bounding box, confidence]	YOLO output of gesture detected	→ M3
Out	M_TargetPosition	Float Array [x, y]	the position target of the user	→ M3, M4

4. Module Interface Specification

Method Name	Inputs	Outputs	Method Description
detectPerson	M_RGB	M_YOLOPerson	Detect presence of a person within processed RGB frame, computes

			bounding box coordinates and confidence score
detectGesture	M_RGB	M_YOLOGesture	Detect presence of hand gesture within processed RGB frame, computes confidence score
detectObjects	M_RGB	M_Objects	Detect presence of other objects in processed RGB frame for the purpose of collision avoidance, computes bounding box coordinates and confidence score
estimateTargetPosition	M_YOLOPerson, M_Depth, M_Scan	M_TargetPosition	Compute position of detected person relative to camera using a blend of information from depth,

			lidar, and processed RGB frame
--	--	--	--------------------------------------

5. Module Internal Design

Block level design

Object detection block

- YOLO detection functions, implemented as matlab function blocks (possible to change due to code generation issues)

Distance and angle calculation

- Matlab function blocks taking in data from bounding boxes, lidar scan, and depth to best approximate current distance of target person from camera. For example, if the height of the user is known, and the focal properties of the camera is known, the distance of the target person can be estimated from the coordinated of the bounding box. Then the three measurements can be blended based on best confidence.

Data Flow

M_RGB -> YOLO person detector -> M_YOLOPerson

M_RGB -> YOLO gesture detector -> M_YOLOGesture

M_RGB -> YOLO object detector -> M_Objects

M_YOLOPerson + M_Scan + M_Depth -> M_TargetPosition

5.3 M3: Visualizer & User Control

1. Functionality

This module serves as the primary Human-Machine Interface (HMI).

- Visualizer: It consumes the robot's perception and state data to render a real-time 3D "Digital Mirror" of the physical twin. Additionally, displays sensor data in real-time to monitor state of the robot.
- Dashboard: It listens for human inputs (clicks, key presses) to send high-level command overrides to the robot.

2. Internal Design

- Hides: Data processing and visualization algorithms, event management for timing of data stream updates

3. Input / Output Variables

I/O	Variable Name	Data Type	Description	Source / Dest
In	M_Scan	Float vector	Scanned Lidar output	← M1
In	M_RGB	Image Vector	RGB Image frame	← M1
In	M_YOLOPerson	Bounding box, Confidence	The YOLO output of detected person	← M2
In	M_Velocity	Double	The monitored velocity	← M1
In	M_Acceleration	Double	The monitored acceleration	← M1
In	M_TargetPosition	Float Vector [x, y]	The Target user position coordinate	← M2
In	M_Objects	Float Vector [x, y]	Position of other objects	← M2
In	M_Battery	Integer	Battery status	← M1
Out	C_User	User input	The human in the loop user interference	User

4. Module Interface Specification

Method Name	Inputs	Outputs	Method Description
displayLidar	M_Scan	Visual Graph or render	Render lidar sensor data to visualize close

			surroundings of the robot
displayRGB	M_RGB	Live camera stream	Display camera feed to visualize pathing and behaviour of the robot
displayMotionState	M_Velocity, M_Acceleration	Visual Graph	Display history of velocity and acceleration
displayTarget	M_TargetPosition	Visual Number	Display position of target person relative to the camera
getUserInput	User keyboard or mouse	C_User	Detect user input to change behaviour mode of the robot

5. Module Internal Design

Block level design

Image display blocks

- Matlab function block to annotate image with corresponding bounding boxes, labels and confidence scores
- Image display block to display annotated images
- Scope block to display velocity, acceleration and battery status
- User input block to change behaviour mode

Data Flow

M_RGB + M_YOLOPerson -> Matlab function block for annotation -> Image display block

M_Scan -> Lidar Plot

M_velocity -> Scope
M_acceleration -> Scope
M_Battery -> Scope
User Input -> C_User

5.4 M4: Navigation

1. Functionality

This module acts as the central decision-making unit. It utilizes a Hybrid Architecture:

- A Finite State Machine (Stateflow) determines the high-level operating mode (Idle, Navigating, Manual) based on gestures and user commands.
- A Deep Reinforcement Learning Agent provides the low-level control policies (velocity/steering) when the system is in the NAVIGATING mode.

2. Internal Design

- Hides: The MATLAB Simulink Model, the neural network policy weights, the reward function logic, and the observation normalization constants.

3. Input / Output Variables

I/O	Variable Name	Data Type	Description	Source / Dest
In	M_TargetPosition	Float Array	Goal location relative to bot.	← M2
In	M_Scan	Float Array	Obstacle distances.	← M1
In	M_YOLOGesture	Integer	Human override commands.	← M2
Out	c_Velocity	Float	Target Speed (m/s).	→ M5
Out	c_Steering	Float	Target Steering Angle (rad).	→ M5

4. Module Interface Specification

Method Name	Inputs	Outputs	Method Description
planMotion	M_TargetPosition, M_scan	c_Velocity, c_Steering	Compute necessary steering angle and speed to follow target person and avoid obstacles. Motion generation is based on a configurable follow-distance setpoint relative to the tracked user.
processGesture	M_YOLOGesture	c_Velocity, c_Steering	Modify motion depending on last valid gesture detected from target person

5. Module Internal Design

Block level design

Target person following blocks

- Matlab function block to compute error between centre of frame and position of target person as well as distance error between desired distance and computed distance to target person.

Obstacle Avoidance

- Use Lidar scan to avoid obstacle and change trajectory if necessary
- Obstacle inputs are evaluated at each control cycle, and stale obstacle data are rejected

Gesture handling

- Use detected gesture and associated confidence score to determine following mode

Data Flow

M_TargetPosition -> Angle and distance error computation block -> c_Velocity + c_Steering

M_Scan -> Matlab function block to determine if collision will occur

-> c_Velocity + c_Steering

M_YOLOGesture -> Confidence threshold block and state transition diagram -> New behaviour state

5.5 M5: Lower-Level Controller

1. Functionality

Executes the velocity and steering requests physically. It runs a PID control loop to ensure the motors spin at the requested speed despite friction or battery voltage changes.

2. Internal Design

- Hides: The PID gains (K_p , K_i , K_d), the specifics of the motor driver PWM frequency, and the dead-zone compensation logic.

3. Input / Output Variables

I/O	Variable Name	Data Type	Description	Source / Dest
In	M_Velocity	Float	Measured speed from sensors(m/s)	← M1
In	c_Velocity	Float	Requested speed (m/s).	← M4
In	c_Steering	Float	Requested angle (rad).	← M4
Out	hw_Motor_PWM	Integer	Voltage signal [0-255].	→ Hardware
Out	hw_Servo_PWM	Integer	Servo pulse width.	→ Hardware

4. Behavior & Range Logic

Input Condition	Output Behavior
c_Velocity >3.0 (Max Limit)	Output: hw_Motor_PWM clamped to max safe value
c_Velocity is very small (<0.01)	Output: hw_Motor_PWM = 0 (Dead-zone handling).
c_Steering out of range (>±1.4 rad/s)	Output: hw_Servo_PWM clamped to physical limits.

$\text{abs}(c_Velocity - M_Velocity)$ (>allowable threshold for more than 0.5 s)	Output: hw_Motor_PWM = 0 to stop the vehicle.
---	---

5. Module Interface specification

Module Name	Inputs	Outputs	Method Description
computeMotorPWM	c_Velocity	hw_Motor_PWM	Convert desired velocity into corresponding PWM signal to control motor
computeServoPWM	c_Steering	hw_Servo_PWM	Convert desired steering angle into corresponding PWM signal to control servo

6. Module Internal Design

Block Level Design

PID Control

- PID Control block for velocity
- PID Control block for steering

PWM Ceiling Limiting

- Ceiling comparison block to limit PWM range of output

Data Flow

c_Velocity -> PID -> Ceiling Check -> hw_Motor_PWM

c_Steering -> PID -> Ceiling Check -> hw_Servo_PWM

6. Detailed Sub-System Design (Hardware/Physical Twin)

6.1 Electrical Schematics

Our project's physics twin is built on the JetAcker platform. The electronic control system follows a two-tier architecture: the upper controller is the Raspberry Pi 5 as the ROS main controller, and the lower controller is the mini STM32 control board as the lower chassis motion controller. STM32 is connected to a drive motor equipped with an encoder and integrated with an IMU, responsible for low-level motion control and sensor data acquisition.

In terms of power transmission, JetAcker's power module accepts 6-14V inputs and provides two 5V outputs and two 12V outputs. In the robot system, the power module input is connected to the battery. The 5V output can power Jetson Nano or Raspberry Pi 5, while the 12V output is used for Jetson Orin series configurations.

The key points of chassis actuators and sensors in electrical integration are summarized as follows:

- (1) Drive motor: The chassis adopts Hall encoder DC gear motor, AB phase pulse output, and the motor working voltage is 12V.
- (2) Steering: The robot uses two HTS-20H bus servo systems, one of which drives the front wheels to turn.
- (3) IMU: The mini STM32 control board integrates a 6-axis IMU connected via I2C.
- (4) Depth Camera: The platform is equipped with the Astra Pro Plus depth camera, which includes an infrared camera, infrared projector, and depth processor.
- (5) LiDAR: This platform is compatible with A1 and G4 LiDAR models; The project ultimately uses G4 LiDAR.

6.2 Bill of Materials (BoM)

This section lists the main hardware elements required for physical twins: the price of the entry-level kit is displayed as \$399.99 on the JetAcker official product page [1]; The chassis/platform adopts JetAcker ROS educational robot automotive platform; The ROS main controller is Raspberry Pi 5, equipped with a ROS robot control board; The low-level motion control is completed by the mini STM32 robot controller board, which is used for chassis low-level control and sensor acquisition; The actuator includes a Hall coded DC gear motor for traction drive and an HTS-20H bus servo for front wheel steering; Aurora 930 Pro structured light depth camera for \$149.99 [3] and IMU integrated on the chassis controller board; The power module includes 6-14V input terminals, multiple 5V and 12V output terminals for system power distribution, and a

battery pack as a configuration component of the JetAcker platform. The product price of SLAMTEC RPLIDAR A1/A2 is \$79.99[2].

6.3 Hardware Abstraction Layer

The Hardware Abstraction Layer (HAL) of the physical twin is responsible for bridging the high-level logic based on ROS running on the ROS main controller (Raspberry Pi 5) and the low-level chassis motion/sensing functions implemented on the mini STM32 controller. In the JetAcker platform, the ROS robot consists of two core components: a chassis, a bottom layer STM32 controller that performs robot motion control and sensor data acquisition, and a ROS main control unit that runs ROS and functional algorithms. By clearly describing the responsibilities of the STM32 firmware and the serial interface used for exchanging commands and telemetry between two levels, the design requirements for "briefly describing the firmware for reading sensors" have been supplemented.

Hardware/connection interface. The standard hardware connection between the ROS main controller and the STM32 chassis controller is achieved through the onboard USB serial port using a power cord and USB serial cable. This physical link is the foundation of HAL: all high-level motion commands and low-level telemetry pass through this serial channel.

Communication abstraction. JetAcker uses a standardized serial communication protocol called RRC communication protocol for programming and communication between upper level machines and lower level STM32 controllers. This protocol uses a hexadecimal based frame format with a two byte frame header followed by a function code, data length, parameters, and CRC checksum to verify the correctness of the data. In our system design, HAL considers this protocol as a low-level transport layer: the ROS side packages commands into RRC frames, and the STM32 side parses the frames and returns telemetry frames. Embedded firmware responsibility. The STM32 board is the underlying motion controller, mainly responsible for motor PID control, encoder and IMU data acquisition, and related low-level drive/sensing tasks; The chassis has been burned with the corresponding code for direct use and supports updates through USB serial or debugging through SWD.

It communicates with the chassis control node of the ROS base layer through a serial port: it receives the target vector velocity from the base layer and sends back real-time speed, IMU data, and battery voltage data calculated by the odometer. The STM32 firmware design uses FreeRTOS as the embedded operating system. Internally, the control principle is to obtain the target speed through inverse kinematics and input it into the motor speed PID controller; After PID calculation, the STM32 timer outputs a PWM motor control signal to the motor driver; The encoder measures the motor speed in real-time and feeds it back to the PID loop to achieve closed-loop speed control. Map to system variables. From the perspective of system level variable models, HAL is an implementation point where high-level controlled variables are transformed into low-level "target vector velocity" and transmitted to the chassis controller,

while low-level telemetry is elevated to monitoring variables for the rest of the system. It also defines the inputs for monitoring, such as battery status and monitored acceleration. Therefore, consistent with the JetAcker interface, HAL sends advanced motion intent as the target vector velocity downstream, as well as publishes upstream telemetry data, including velocity, IMU related data, and the relationship between battery voltage of advanced modules and initialization and unexpected events. The system design documents explicitly require handling of unwanted events, such as communication loss and sensor failures. At the HAL boundary, the key hardware level prerequisite for normal operation is the standard power supply and USB serial connection between the ROS main controller and STM32 controller. The detailed adverse event handling strategy is recorded in the dedicated "Adverse Event Handling" section of the system design.

7. Traceability Matrix

Requirement ID(s)	Stakeholder	Needs	Deliverable
REQ-SB-01, REQ-SB-02	Safety	Immediate safety	M4: zero-velocity policy in Initial and Idle. M5: zero motor output
REQ-SB-03	User	Smooth following	M4: follow-distance control logic
REQ-SB-04, REQ-SB-07	User	Responsiveness	M2: gesture recognition M4: gesture-action gating/ state-dependent gating
REQ-SB-05	Safety	Control stability	M5: linear/angular command clamping
REQ-SB-06, REQ-GSB-01, REQ-GSB-04	User	Direct control	M3: User interface for manual input path M4: manual routing M5: execution
REQ-SB-08, REQ-SB-09	Safety	Perception reliability	M2: tracked user only gating + confidence gesture selection
REQ-SB-10	User	Adjustable following distance	M4: follow distance setpoint update logic
REQ-SB-11	Safety	Conservative navigation	M2: obstacle interpretation M4: clearance-aware motion planning

REQ-ST-01, REQ-ST-02, REQ-ST-03	Safety	Safe degradation	M4: initial state guards and startup transition conditions
REQ-ST-04, REQ-ST-05	System	Reliability	M4: follow state transition guards
REQ-ST-06, REQ-ST-07	User	Fast reaction	M4: manual override transition path M3: user interface support for manual-mode state change
REQ-ST-08, REQ-ST-09, REQ-ST-10	System	Reliability	M4: Simulink manual-exit transition module
REQ-GSB-02	Safety	Ultimate safety	M3/M4: emergency-stop override M5: zero motor output
REQ-GSB-03	User	Flexible operation	M4: runtime mode-switching architecture M3: DT high-level command interface
REQ-SM-01, REQ-SM-03	System	Verification & debugging	M1–M3: telemetry mirroring to DT+ computational offloading
REQ-SM-02	User	Supervisory control	M3: DT high-level command interface

Note: The requirements that are highlighted in red are the ones that are likely to be changed as we continue the development and the testing process if the system reaches its software or hardware bottleneck.

8. Undesired Event Handling

This section describes what happens if Wi-Fi disconnects, camera is occluded and Simulink model crashes during the operation. The main goal is to ensure the vehicle changes to a STOP or safe pending state when safe autonomous operation cannot be guaranteed.

8.1 Normal Behaviour

Under normal operating conditions, the system operates in either Human-Following Mode or Manual Control mode.

In Human-Following mode, the vehicle continuously detects and tracks a designated target

person and maintains a desired following distance through closed-loop control. Obstacle information from onboard sensors is incorporated in real time to ensure the safety.

In Manual Control Mode, the user commands, such as hand gestures or GUI inputs, are used to switch modes or adjust following parameters. During normal operation, all motion commands remain within predefined safety limits, and the system behavior is continuously observable through the dashboard.

8.2 Wi-Fi Disconnects

1. When the connection between the physical twin and the digital twin is lost, the vehicle immediately changes to a STOP state.
2. Set the linear and angular velocity to zero and activate the safety stop ($C_Stop = 1$).
3. A warning message ("*Wi-Fi Disconnected*") is displayed on the dashboard to inform the user.
4. Normal operation may resume only after communication is restored and the system is confirmed to be safe by the user.

8.3 Camera Occluded

1. If the camera is occluded, the system will disable the functions of person following and gesture-based mode switching.
2. During the Follow Mode, the vehicle changes to a Target Lost/STOP state, decelerates smoothly and comes to a complete stop.
3. For the path planning, the vehicle does not move toward the last predicted target to prevent unsafe motions.
3. A warning message ("*Target Lost*") is displayed on the dashboard to inform the user.
4. The system remains in the Target Lost/STOP state until visual information is restored, and the target person is re-identified with sufficient confidence.

8.4 Simulink Model Crashes

1. If the Simulink model crashes, the system immediately enters a STOP state.
2. All motion outputs are set to safe values such as zero velocity to ensure the vehicle does not continue moving.
3. A warning message ("*Controller Error*") is displayed on the dashboard to inform the user.
4. Manual action such as restarting the vehicle is needed, followed by a successful system check before resuming operation.

5. Record the fault code, environment (some pictures from the camera) and time in the log for future system optimization and reducing the occurrence of crashes.

9. Appendices

- A. References
- B. Preliminary Code/Diagrams

[1] “Hiwonder Jetacker AI Robot Kit – nvidia jetson-powered ros1/Ros2 Educa,” Hiwonder, <https://www.hiwonder.com/products/jetacker?variant=40227272917079> (accessed Dec. 19, 2025).

[2] “SLAMTEC RPLIDAR Lidar C1 A1 A2M12 S2L slam triangulation ranging suppo,” Hiwonder, <https://www.hiwonder.com/products/lidar-a1-a2?variant=40471821516887> (accessed Dec. 19, 2025).

[3] “Hiwonder AURORA930 Pro 3D Structured Light Depth Camera 3D Mapping & R,” Hiwonder, https://www.hiwonder.com/products/aurora930-pro?srsId=AfmBOomyZYyaaol16e3J6dZUcvb0mbZy6bENak_Pzxz1Q9Xqxm9lGZ (accessed Dec. 19, 2025).